

Aira Samson -166456

Njugi Elijah –146412

Kihumba Kevin-168377

Simulation of a Banking Queue System Using Java

1.1 Introduction

Computer simulation and modelling is a technique used to imitate the operation of real-life systems to study their behaviour under different conditions. In service systems such as banks, simulation is useful in analysing customer arrivals, waiting times, service times and server utilization. By simulating the queue, it becomes possible to estimate how efficiently the system is performing and whether customers are experiencing long delays.

In this project, a single-server banking system was simulated using Java. The system assumes that customers arrive at the bank at random intervals and are served by one teller. The inter-arrival times of customers are uniformly distributed between 1 and 8 minutes, while the service times are uniformly distributed between 1 and 6 minutes. The simulation was carried out for 100 customers, and queue statistics such as average waiting time, average service time, average time in the system, maximum waiting time, server utilization, and idle time were computed.

1.2 Problem Statement

A banking system is to be simulated for 100 customers. The arrival times are assumed to be uniformly distributed on the interval (1, 8) minutes, while the service times are uniformly distributed on the interval (1, 6) minutes. The objective is to model the system and determine queue performance measures such as waiting time, service time, server utilization, and total simulation time.

1.3 Objectives of the System

- i. To simulate a banking queue system for 100 customers.
- ii. To generate customer inter-arrival times uniformly between 1 and 8 minutes.
- iii. To generate service times uniformly between 1 and 6 minutes.
- iv. To compute customer arrival times, service start times, service end times, and waiting times.
- v. To calculate queue statistics such as:

average waiting time

average service time

average time in system

maximum waiting time

probability that a customer waits

server utilization

total idle time of the server

total simulation time

1.4 Assumptions of the Model

The following assumptions were made in the simulation:

1. The bank has one teller/server.
2. Customers are served on a First-Come First-Served (FCFS) basis.
3. Inter-arrival times are uniformly distributed between 1 and 8 minutes.
4. Service times are uniformly distributed between 1 and 6 minutes.
5. The queue has unlimited capacity.
6. Every customer who arrives joins the queue and waits until served.
7. No customer leaves the queue before being served.
8. The simulation starts with an empty queue and an idle teller.
9. The simulation ends after the 100th customer is served.

1.5 Methodology

A discrete-event simulation approach was used to model the banking queue system. The simulation tracks each customer from the time of arrival up to the time service is completed. For each of the 100 customers, a random inter-arrival time and a random service time were generated. Using these values, the arrival time, service start time, waiting time, service completion time, and time spent in the system were calculated.

The arrival time of each customer was determined cumulatively from the inter-arrival times. The service start time was obtained by comparing the customer's arrival time with the completion time of the previous customer. If the teller was free when the customer arrived, service began immediately; otherwise, the customer waited in the queue until the teller became available. At the end of the simulation, the performance of the banking system was evaluated using queue statistics such as average waiting time, average service time, maximum waiting time, probability of waiting, server utilization, total idle time, and total simulation time.

1.6 Simulation Algorithm

- i. The following steps were followed in the simulation:
- ii. Set the number of customers to 100.
- iii. Generate random inter-arrival times between 1 and 8 minutes.
- iv. Generate random service times between 1 and 6 minutes.
- v. Compute the arrival time of each customer.
- vi. For the first customer:

set service start time equal to arrival time

waiting time = 0

service end time = service start time + service time

- vii. For each remaining customer:

compute service start time as the maximum of arrival time and previous customer's service end time

compute waiting time as service start time minus arrival time

compute service end time

compute total time in system

compute idle time of the server if applicable

- viii. Sum all waiting times, service times, and idle times.
- ix. Determine the maximum waiting time.
- x. Compute average waiting time, average service time, average time in system, probability of waiting, server utilization, and total simulation time.
- xi. Display the simulation table and the final queue statistics.

1.7 Mathematical Formulation

Arrival Time

For the first customer:

$$A_1 = IA_1$$

For the remaining customers:

$$A_i = A_{i-1} + IA_i$$

Where:

- A_i = arrival time of customer i
- IA_i = inter-arrival time of customer i
- **Service Start Time**

$$SS_i = \max (A_i, SE_{i-1})$$

Where:

- SS_i = service start time of customer i
- SE_{i-1} = service end time of previous customer

Waiting Time

$$W_i = SS_i - A_i$$

Where:

- W_i = waiting time of customer i

Service End Time

$$SE_i = SS_i + ST_i$$

Where:

- ST_i = service time of customer i

Time in System

$$T_i = W_i + ST_i$$

Where:

- T_i = total time customer i spends in the system

Server Utilization

$$\text{Utilization} = \frac{\text{Total Service Time}}{\text{Total Simulation Time}} \times 100$$

1.8 Java Program Used for the Simulation

```
import java.util.Random;

public class BankSimulation {
    public static void main(String[] args) {

        final int NUM_CUSTOMERS = 100;
        Random rand = new Random();

        int[] interArrival = new int[NUM_CUSTOMERS];
        int[] arrivalTime = new int[NUM_CUSTOMERS];
        int[] serviceTime = new int[NUM_CUSTOMERS];
        int[] serviceStart = new int[NUM_CUSTOMERS];
        int[] serviceEnd = new int[NUM_CUSTOMERS];
        int[] waitingTime = new int[NUM_CUSTOMERS];
        int[] timeInSystem = new int[NUM_CUSTOMERS];
        int[] idleTime = new int[NUM_CUSTOMERS];

        int totalWaiting = 0;
        int totalService = 0;
        int totalSystemTime = 0;
        int totalIdle = 0;
        int maxWaiting = 0;
        int customersWhoWaited = 0;

        // -----
        // STEP 1: Generate inter-arrival and service times
        // Inter-arrival ~ Uniform(1,8)
        // Service time ~ Uniform(1,6)
        // -----
        for (int i = 0; i < NUM_CUSTOMERS; i++) {
            interArrival[i] = rand.nextInt(8) + 1; // 1 to 8
            serviceTime[i] = rand.nextInt(6) + 1; // 1 to 6
        }

        // -----
        // STEP 2: Compute arrival times
        // -----
        arrivalTime[0] = interArrival[0];
        for (int i = 1; i < NUM_CUSTOMERS; i++) {
            arrivalTime[i] = arrivalTime[i - 1] + interArrival[i];
        }

        // -----
        // STEP 3: First customer
        // -----
        serviceStart[0] = arrivalTime[0];
        waitingTime[0] = 0;
        idleTime[0] = arrivalTime[0]; // server idle before first customer
    }
}
```

```

arrives
    serviceEnd[0] = serviceStart[0] + serviceTime[0];
    timeInSystem[0] = waitingTime[0] + serviceTime[0];

    totalService += serviceTime[0];
    totalSystemTime += timeInSystem[0];
    totalIdle += idleTime[0];

    // -----
    // STEP 4: Remaining customers
    // -----
    for (int i = 1; i < NUM_CUSTOMERS; i++) {
        // Service starts either when customer arrives or when previous
service ends
        serviceStart[i] = Math.max(arrivalTime[i], serviceEnd[i - 1]);

        // Waiting time in queue
        waitingTime[i] = serviceStart[i] - arrivalTime[i];

        // Idle time of server before this customer starts
        if (arrivalTime[i] > serviceEnd[i - 1]) {
            idleTime[i] = arrivalTime[i] - serviceEnd[i - 1];
        } else {
            idleTime[i] = 0;
        }

        // Service completion time
        serviceEnd[i] = serviceStart[i] + serviceTime[i];

        // Total time in system
        timeInSystem[i] = waitingTime[i] + serviceTime[i];

        // Totals
        totalWaiting += waitingTime[i];
        totalService += serviceTime[i];
        totalSystemTime += timeInSystem[i];
        totalIdle += idleTime[i];

        // Maximum waiting time
        if (waitingTime[i] > maxWaiting) {
            maxWaiting = waitingTime[i];
        }

        // Count customers who had to wait
        if (waitingTime[i] > 0) {
            customersWhoWaited++;
        }
    }

    // -----
    // STEP 5: Compute statistics
    // -----
    double avgWaiting = (double) totalWaiting / NUM_CUSTOMERS;
    double avgService = (double) totalService / NUM_CUSTOMERS;
    double avgSystem = (double) totalSystemTime / NUM_CUSTOMERS;
    double probabilityWait = (double) customersWhoWaited /
NUM_CUSTOMERS;
    int simulationTime = serviceEnd[NUM_CUSTOMERS - 1];
    double utilization = ((double) totalService / simulationTime) *
100.0;

```

```

// -----
// STEP 6: Print simulation table
// -----
System.out.println("===== BANK QUEUE
SIMULATION =====");
System.out.printf("%-6s %-12s %-10s %-10s %-12s %-10s %-10s %-12s
%-10s\n",
    "Cust", "InterArr", "Arrival", "Service", "StartServ",
"EndServ", "Wait", "TimeSys", "Idle");

    for (int i = 0; i < NUM_CUSTOMERS; i++) {
        System.out.printf("%-6d %-12d %-10d %-10d %-12d %-10d %-10d %-
12d %-10d\n",
            (i + 1),
            interArrival[i],
            arrivalTime[i],
            serviceTime[i],
            serviceStart[i],
            serviceEnd[i],
            waitingTime[i],
            timeInSystem[i],
            idleTime[i]);
    }

// -----
// STEP 7: Print summary statistics
// -----
System.out.println("\n===== QUEUE STATISTICS
=====");
    System.out.println("Number of Customers          : " +
NUM_CUSTOMERS);
    System.out.printf("Average Waiting Time          : %.2f minutes\n",
avgWaiting);
    System.out.printf("Average Service Time          : %.2f minutes\n",
avgService);
    System.out.printf("Average Time in System          : %.2f minutes\n",
avgSystem);
    System.out.println("Maximum Waiting Time          : " + maxWaiting
+ " minutes");
    System.out.printf("Probability Customer Waits          : %.2f\n",
probabilityWait);
    System.out.printf("Server Utilization          : %.2f%%\n",
utilization);
    System.out.println("Total Idle Time of Server          : " + totalIdle +
" minutes");
    System.out.println("Total Simulation Time          : " +
simulationTime + " minutes");
}
}

```

1.9 Simulation Results

```
"C:\Program Files\Java\jdk-20\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2023.2.1\lib\idea_rt.jar=6202..."
```

===== BANK QUEUE SIMULATION =====

Cust	InterArr	Arrival	Service	StartServ	EndServ	Wait	TimeSys	Idle
1	8	8	6	8	14	0	6	8
2	1	9	4	14	18	5	9	0
3	6	15	4	18	22	3	7	0
4	5	20	1	22	23	2	3	0
5	1	21	2	23	25	2	4	0
6	4	25	4	25	29	0	4	0
7	4	29	2	29	31	0	2	0
8	1	30	5	31	36	1	6	0
9	7	37	6	37	43	0	6	1
10	3	40	4	43	47	3	7	0
11	7	47	5	47	52	0	5	0
12	2	49	1	52	53	3	4	0
13	1	50	3	53	56	3	6	0
14	8	58	4	58	62	0	4	2
15	2	60	4	62	66	2	6	0
16	8	68	5	68	73	0	5	2
17	6	74	2	74	76	0	2	1
18	3	77	6	77	83	0	6	1
19	7	84	3	84	87	0	3	1

Run BankSimulation x

↑	81	1	354	1	362	363	8	9	0
↓	82	1	355	2	363	365	8	10	0
↔	83	6	361	1	365	366	4	5	0
↕	84	8	369	1	369	370	0	1	3
↖	85	4	373	6	373	379	0	6	3
↗	86	5	378	4	379	383	1	5	0
↘	87	2	380	2	383	385	3	5	0
↙	88	4	384	3	385	388	1	4	0
	89	3	387	3	388	391	1	4	0
	90	2	389	6	391	397	2	8	0
	91	8	397	6	397	403	0	6	0
	92	8	405	6	405	411	0	6	2
	93	7	412	1	412	413	0	1	1
	94	2	414	1	414	415	0	1	1
	95	1	415	4	415	419	0	4	0
	96	7	422	5	422	427	0	5	3
	97	8	430	5	430	435	0	5	3
	98	4	434	1	435	436	1	2	0
	99	8	442	3	442	445	0	3	6
	100	1	443	2	445	447	2	4	0

===== QUEUE STATISTICS =====

Number of Customers : 100


```
===== QUEUE STATISTICS =====
Number of Customers      : 100
Average Waiting Time     : 1.91 minutes
Average Service Time     : 3.59 minutes
Average Time in System   : 5.50 minutes
Maximum Waiting Time     : 10 minutes
Probability Customer Waits : 0.55
Server Utilization       : 80.31%
Total Idle Time of Server : 88 minutes
Total Simulation Time     : 447 minutes

Process finished with exit code 0
```

Queue Statistic	Result
Number of customers	100
Average waiting time	1.91 minutes
Average service time	3.59 minutes
Average time in system	5.50 minutes
Maximum waiting time	10 minutes
Probability customer waits	0.55
Server utilization	80.31%
Total idle time of server	88 minutes
Total simulation time	447 minutes

1.10 Discussion of Results

The simulation of the banking queue system for 100 customers showed that the average waiting time was 1.91 minutes, indicating that customers generally spent a short time waiting in the queue before being served, while the average service time was 3.59 minutes, which is consistent with the service time distribution of 1 to 6 minutes. The average time spent in the system was 5.50 minutes, meaning that a customer spent approximately 5.50 minutes in the bank from arrival to departure, including both waiting and service time. The maximum waiting time observed was 10 minutes, showing that although most customers were served quickly, a few experienced longer delays when the teller was busy. The probability that a customer waits was 0.57, which means that 57% of the customers had to wait before being served, while 43% were served immediately upon arrival. In addition, the server utilization was 80.31%,

indicating that the teller was busy for most of the simulation period but was not overloaded, which suggests that the system remained stable and reasonably efficient. The total idle time of the server was 88minutes, representing the periods when no customer was being served, and the total simulation time was 447 minutes, which is the time taken for all 100 customers to be completely served.